COMS 6998 — High Performance Machine Learning

Homework Assignment 2: Profiling Small LLM Workloads

Instructor: Dr. Kaoutar El Maghraoui *Due: October 19, 2025*
Fall 2025 *Max Points: 100*

Overview & Objectives

This assignment focuses on profiling and analyzing the performance of *small language model (LLM)* workloads in PyTorch. You will fine-tune a lightweight transformer (e.g., DistilBERT) on a text classification dataset, vary key hyperparameters, and use the **PyTorch Profiler** to understand where time is spent (data loading, forward, backward, optimizer). You will also evaluate `torch.compile` (Inductor backend).

Deliverable: a single Jupyter notebook (`.ipynb`) that runs top-to-bottom and contains:

- All code for C1–C8 (and C9 if attempted).
- Required **tables/figures** listed under each component below.
- PyTorch Profiler outputs (text table screenshots or copied summaries) and TensorBoard/Chrome traces when applicable.
- Short answers for Q1–Q5.
- Link and screenshot of your Weights & Biases (W&B) run.

Epochs. Train for **5 epochs by default**. You may optionally run up to **10 epochs** to observe convergence; optional epochs are not required for full credit.

Computing Environment

Use either **Google Colab (GPU)** or the **Columbia Insomnia cluster**. At the top of your notebook, print and state:

- GPU model (if any), CPU model (optional).
- PyTorch & Transformers versions.
- CUDA version (if applicable).

Experiment Tracking (Weights & Biases)

We will use **Weights & Biases (W&B)** for experiment tracking. You may log in or run anonymously.

Your notebook must:

1. Initialize a W&B run and record a **config** with at least: model name, sequence length, batch size, learning rate, optimizer, number of dataloader workers, epochs, device (CPU/GPU), and whether `torch.compile` is enabled.

2. **Log per-epoch metrics:** training loss, training accuracy, test accuracy, data-loading time, compute time (forward+backward+optimizer), and total epoch time.
3. **Name/group runs** meaningfully (e.g., `bs_32_lr_1e-4`, group: `optimizer_compare`).
4. For sweeps (C3/C6/C7/C8), **log a results table** (Pandas → W&B Table) and optionally **upload artifacts** (CSV or plot images).
5. Include a link or screenshot of your W&B run page.

Minimal snippet (in your notebook):

```
# %pip install wandb
import os, wandb
wandb.init(project="hpml-hw2-llm")
wandb.config.update({
  "model_name": "distilbert-base-uncased",
  "max_len": 256, "batch_size": 32, "lr": 1e-4,
  "optimizer": "AdamW", "num_workers": 2,
  "epochs": 5, "compile_mode": False
})
# Per epoch: wandb.log({"train/loss": ..., "time/epoch": ..., "test/acc": ...})
```

Privacy. Log only training metadata/metrics (no raw IMDB text).

Dataset & Model

- **Dataset:** IMDB sentiment classification (binary), via HuggingFace `datasets`.
- **Model:** `DistilBERTForSequenceClassification`.
- **Tokenization:** Use the appropriate tokenizer and *dynamic padding* in a custom `collate_fn`.
- **Sequence length:** Use a fixed max length (e.g., 256) across runs you intend to compare.

Coding Exercises & Required Result Formats (100 pts total)

Unless specified, assume 5 training epochs and device-agnostic code (CPU/GPU). For each component, produce the exact result formats below.

C1 (15 pts): Fine-tuning a Small LLM

Task: Load IMDB, tokenize, and fine-tune the model for 5 epochs (optionally up to 10).

- **Figure F1**: Line plot of *training loss* and *training accuracy* vs. epoch (same figure with two y-axes or two subplots).
- **Table T1**: Per-epoch metrics with columns:

Epoch	Train Loss	Train Acc	Test Acc
1			
⋮			
5			

C2 (10 pts): Baseline Timing

Task: Measure for each epoch: (a) data-loading time, (b) training compute time (forward+backward+optimizer), (c) total epoch time. Ensure CUDA synchronization around timing as needed.

- **Table T2** (per epoch):

Epoch	Data Time (s)	Compute Time (s)	Total (s)	Device
1				
⋮				
5				

- **Figure F2**: Stacked bar chart per epoch showing data vs. compute contribution to total time.

C3 (10 pts): DataLoader Performance

Task: Vary `num_workers` $\in \{0, 2, 4, 8\}$ (keep other settings fixed).

- **Table T3**:

num_workers	Avg Epoch Time (s)	Avg Data Time (s)	Batch Size
0			
2			
4			
8			

- **Figure F3**: Line plot of *avg epoch time* vs. *num_workers*.
- **One-sentence conclusion**: Which `num_workers` is optimal and why?

C4 (15 pts): PyTorch Profiler

Task: Use `torch.profiler` to generate a trace with `num_workers=1` and with the optimal value found in C3. Report percentage of time in data loading vs. forward vs. backward vs. optimizer.

- **Table T4** (two rows: `nw=1` and `nw=best`):

Setting	Data (%)	Forward (%)	Backward (%)	Optimizer (%)
nw=1				
nw=best				

- **Figure F4**: Screenshot or saved table of top ops by *self_cpu_time_total* (and GPU if available).

C5 (10 pts): CPU vs. GPU

Task: Train for 5 epochs on CPU and on GPU. Keep batch size and max length fixed across both.

- **Table T5**:

Device	Avg Epoch Time (s)	Final Train Loss	Final Test Acc
CPU			
GPU			

- **Figure F5**: Bar chart comparing *avg epoch time* CPU vs. GPU.

C6 (15 pts): Hyperparameter Sensitivity

Task: Sweep Batch size $\in \{16, 32, 64\}$ and Learning rate $\in \{5 \times 10^{-5}, 10^{-4}, 5 \times 10^{-4}\}$.

- **Table T6** (one row per config):

Batch	LR	Avg Epoch Time (s)	Final Train Loss	Train Acc	Test Acc

- **Figure F6a**: Heatmap of *avg epoch time* by (batch, lr).
- **Figure F6b**: Heatmap of *test accuracy* by (batch, lr).
- **2–3 sentence discussion** of speed vs. accuracy vs. stability trade-offs.

C7 (10 pts): Optimizer Comparison

Task: Train for 5 epochs with **SGD**, **Adam**, and **AdamW** (same batch size/lr unless optimizer requires adjustment).

- **Table T7**:

Optimizer	Avg Epoch Time (s)	Final Train Loss	Train Acc	Test Acc
SGD				
Adam				
AdamW				

- **Figure F7**: Bar chart of *avg epoch time* by optimizer.
- **2 sentences**: convergence/variance observations.

C8 (10 pts): `torch.compile`

Task: Compare Eager vs. `torch.compile(..., backend="inductor")` with 10 total epochs. Report first-epoch time and average across epochs 6–10. (Epochs 1–5 serve as warmup.)

- **Table T8**:

Mode	First Epoch Time (s)	Avg Time Epochs 6–10 (s)
Eager		
Compile (Inductor)		

- **Figure F8**: Side-by-side bars for first-epoch time; side-by-side bars for avg(6–10).
- **1–2 sentences**: why the first compiled epoch is slower and later epochs faster.

C9 (Extra Credit, 10 pts): Advanced Profiling

Task: Generate an operator-level trace with `torch.profiler` and identify at least two bottlenecks. Propose concrete optimizations (e.g., dataloader tuning, mixed precision where appropriate, sequence length/batch size trade-offs, avoiding CPU↔GPU syncs).

- **Figure F9**: Screenshot of TensorBoard/Chrome trace (or profiler top-ops table).
- **Bullet list**: 2+ bottlenecks and specific mitigations.

Short Answer Questions (20 pts)

Q1. (3 pts) What is the input dimension of DistilBERT's embedding layer?

Q2. (3 pts) What is the output dimension of the classifier head for IMDB?

Q3. (8 pts) Report the number of trainable parameters and parameters with gradients after a backward pass. Include the code you used to compute both.

Q4. (2 pts) How (if at all) does parameter count change when switching from SGD to Adam?

Q5. (4 pts) Why is the first epoch slower with `torch.compile`, and why are later epochs typically faster?

Submission Instructions

Submit a **single Jupyter notebook** (`.ipynb`) that includes:

- Code and outputs for C1–C8 (and C9 if attempted).
- All required **tables T1–T8** and **figures F1–F8** (and F9 if doing extra credit).
- Profiler summaries (table text or screenshots) and optional trace files (as artifacts via W&B).
- Clear environment summary (device, versions) at the top.
- W&B link/screenshot for your main runs and sweeps.

Your notebook **must** execute cleanly top-to-bottom without manual intervention.

Grading Rubric (100 pts)

Component	Points
C1: Fine-tuning	15
C2: Baseline timing	10
C3: DataLoader performance	10
C4: PyTorch Profiler	15
C5: CPU vs. GPU	10
C6: Hyperparameter sensitivity	15
C7: Optimizer comparison	10
C8: `torch.compile`	10
Short answers (Q1–Q5)	15
Total	**100**

Implementation Notes & Tips

- Use dynamic padding in `collate_fn` for efficiency and stable memory.
- Synchronize CUDA around timing (`torch.cuda.synchronize()`) for accurate GPU timings.
- Keep sequence length fixed across compared runs; change *one* factor at a time when profiling.
- When memory is tight, reduce batch size first; avoid changing multiple variables simultaneously.
- For reproducibility: set seeds for Python, NumPy, and PyTorch (CPU & CUDA).